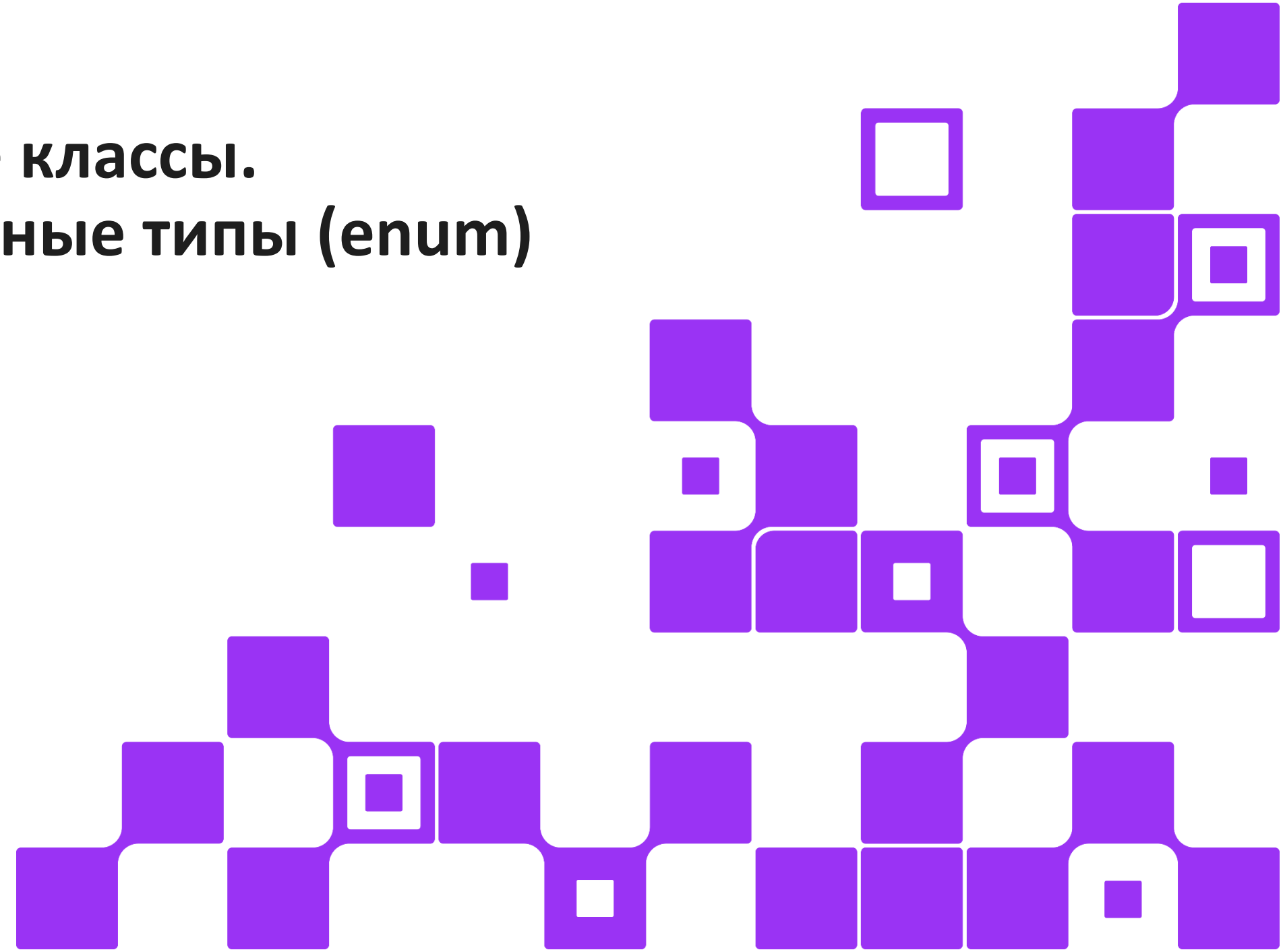
Анонимные классы.
Перечисленные типы (enum)



IThub

Анонимные классы

Анонимный класс (anonymous class) — это класс, у которого:

- нет явного имени;
- он объявляется и создаётся одновременно;
- обычно используется, когда нужен небольшой одноразовый объект с особым поведением.

IThub

```
Runnable runnable = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello!");
    }
};
```

Мы фактически говорим Java:
«Создай объект некоторого класса, который реализует Runnable, и вот прямо здесь определи, как он должен работать».

У этого класса нет имени вроде:
`class MyRunnable implements Runnable {...}`

IThub

Анонимный класс может реализовать интерфейс

Это один из самых распространённых вариантов.

```
interface Greeting {
    void sayHello();
}
```

```
Greeting greeting = new Greeting() {

    @Override
    public void sayHello() {
        System.out.println("Hello!");
    }
};
```

IThub

```
class Animal {

    public void makeSound() {
        System.out.println("Some sound");
    }
}
```

Анонимный класс может наследоваться от класса.

Не обязательно реализовывать интерфейс. Можно создать анонимный наследник обычного класса.

```
Animal animal = new Animal() {

    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }
};
```

=

```
class SomeAnonymousAnimal extends Animal {

    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }
}
```

```
animal.makeSound();
```

```
Woof!
```

IThub

Анонимный класс может иметь собственные поля

```
Animal animal = new Animal() {

    private String name = "Barsik";

    @Override
    public void makeSound() {
        System.out.println(name + ": Meow!");
    }
};
```

IThub

Можно добавлять собственные методы

```
Animal animal = new Animal() {

    @Override
    public void makeSound() {
        System.out.println("Meow");
    }

    public void sleep() {
        System.out.println("Sleeping...");
    }
};
```

Мы можем вызвать animal.makeSound().
Но animal.sleep() не скомпилируется.
Потому что тип переменной Animal animal а в классе Animal метода sleep() нет.
Можно вызвать такой метод внутри самого анонимного класса.

Enum

Перечисленный тип, **перечисления** также называемый **enum** – это тип данных, который состоит из фиксированного набора констант.
Каждая константа в перечисленном типе имеет имя и значение, и они могут использоваться для описания ограниченного набора значений, например, месяцев года, дней недели или типов данных.
Enum **не** может наследоваться от другого класса.

IThub

```
enum Day {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
}
```

```
Day day = Day.MONDAY;
```

enum позволяет выразить ограниченное множество допустимых значений на уровне типов.

Зачем он нужен?

Представьте, что без enum мы решили хранить статус заказа:
String status = "CREATED";

Проблема в том, что кто-то может написать:
String status = "CRETATED";

Опечатка. Компилятор ничего не скажет.

```
enum OrderStatus {
    CREATED,
    PAID,
    SHIPPED,
    DELIVERED,
    CANCELLED
}
```

```
OrderStatus status = OrderStatus.CREATED;

OrderStatus status = OrderStatus.CRETATED;
```

Главная идея enum: ограничить множество допустимых значений

IThub

IThub

enum — это фактически специальный класс. Когда мы пишем:

```
enum Color {
RED,
GREEN,
BLUE
}
```

можно концептуально думать примерно так:

```
Color
├── RED
├── GREEN
└── BLUE
```

RED, GREEN, BLUE — объекты Color.

```
Color color = Color.RED;
```

Означает, что переменная color содержит ссылку на объект Color.RED

У enum есть методы

- **values()**

```
enum Color {
    BLUE,
    RED,
    GREEN
}
```

```
for (Color color : Color.values()) {
    System.out.println(color);
}
```

```
RED
GREEN
BLUE
```

IThub

- **valueOf()**

```
enum Color {
   BLUE,
   RED,
   GREEN
}
```

```
Color color = Color.valueOf("RED");
```

Color.RED

```
Color.valueOf("red");
```

IllegalArgumentException

valueOf() чувствителен к регистру!

IThub

- **Switch и enum**

```
enum OrderStatus {
    CREATED,
    PAID,
    SHIPPED,
    DELIVERED
}
```

```
OrderStatus status = OrderStatus.PAID;

switch (status) {

    case CREATED:
        System.out.println("Order created");
        break;

    case PAID:
        System.out.println("Order paid");
        break;

    case SHIPPED:
        System.out.println("Order shipped");
        break;

    case DELIVERED:
        System.out.println("Order delivered");
        break;
}
```

enum может иметь поля

```
enum HttpStatus {

    OK(200),
    NOT_FOUND(404),
    INTERNAL_SERVER_ERROR(500);

    private final int code;

    HttpStatus(int code) {
        this.code = code;
    }

    public int getCode() {
        return code;
    }
}
```

```
HttpStatus status = HttpStatus.NOT_FOUND;

System.out.println(status.getCode());
```

```
404
```

То есть enum может содержать:

- поля
- конструктор
- методы
- логику

IThub

enum может реализовывать интерфейсы

```
enum Status implements Describable {

    CREATED,
    PAID,
    SHIPPED;

    @Override
    public String getDescription() {

        return switch (this) {
            case CREATED -> "Order created";
            case PAID -> "Order paid";
            case SHIPPED -> "Order shipped";
        };
    }
}
```

```
interface Describable {
    String getDescription();
}
```

```
Describable status = Status.PAID;

System.out.println(status.getDescription());
```

```
enum Country {

    RUSSIA("RU", "Russia"),
    GERMANY("DE", "Germany"),
    FRANCE("FR", "France");

    private final String code;
    private final String displayName;

    Country(String code, String displayName) {
        this.code = code;
        this.displayName = displayName;
    }

    public String getCode() {
        return code;
    }

    public String getDisplayName() {
        return displayName;
    }
}
```

enum может иметь сложные поля

```
Country country = Country.GERMANY;

System.out.println(country.getCode());
System.out.println(country.getDisplayName());
```

```
DE
Germany
```

IThub

Задача: система уведомлений

У нас есть три способа отправки уведомления:

- Email
- Sms
- Push

есть интерфейс:

```
interface NotificationSender {
    void send(String message);
}
```

Напишите метод:

`public static NotificationSender createSender(NotificationType type)`

который возвращает подходящий NotificationSender

Должно получиться:

- Sending EMAIL: Привет!
- Sending SMS: Привет!
- Sending PUSH: Привет!

Для создания NotificationSender используй именно анонимные классы.

```
public class Main {

    enum NotificationType {
        // код
    }

    interface NotificationSender {
        // код
    }

    public static NotificationSender createSender(NotificationType type) {
         switch (type) {
                case EMAIL: return new NotificationSender() {
                    @Override public void send(String message) {
                            ….


    public static void main(String[] args) {

        NotificationSender sender = createSender(NotificationType.EMAIL);
        sender.send("Привет!");
    }
}
```